Автономная некоммерческая организация профессиональная образовательная организация Московский Международный Колледж Цифровых технологий "Академия ТОП"

КАФЕДРА «РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ»

Интеграция модулей маркетинговой платформы "ПромоМакс"

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к учебной практике

Автор работы: студент группы 9/3-РПО-23/1 Митин П.А.

Руководитель: доц. Гвоздев С. М.

Работа защищена: _____ оценка_____

Члены комиссии: _____

_____

_____

Новомосковск 2026

Оглавление

ВВЕДЕНИЕ..... 3
ГЛАВА 1. Проектирование пользовательского интерфейса и архитектуры…….…..….4
1.1. Создание макетов в Figma…….....…….4
1.2. Структура экранов и навигация……..……7
1. 3. Проектирование модуля аналитики…….....…….8
1. 4. Проектирование модуля интеграций……..……9
ГЛАВА 2. Выбор технологий и инструментов разработки.....…….……...10
2.1. Платформа и фреймворки…….....……...10
2.2. Библиотеки для визуализации и экспорта…….....……...10
2.3. Работа с базой данных и шифрованием…….....……...11
ГЛАВА 3. Разработка модуля анализа эффективности…….....……...12
3.1. Сервис аналитики (AnalyticsService)……..……..……...12
3.2. Реализация линейного прогноза тренда (МНК) …….....……...14
3.3. A/B тестирование кампаний…….....……...14
3.4. ViewModel для дашборда (DashboardViewModel) …….....15
3.5. ViewModel для аналитики кампании (CampaignAnalyticsViewModel) ……..16
3.6. Построение графиков с LiveCharts…….....……...19
ГЛАВА 4. Интеграция модулей: настройка автоматического обмена…….……...21
4.1. Сервис вебхуков (WebhookService) …….....…….……...21
4.2. Шифрование API-ключей (EncryptionService) …….....……...22
4.3. Логирование обменов и мониторинг…….....……...22
4.4. Интерфейс управления интеграциями (IntegrationsViewModel) ……23
ГЛАВА 5. Тестирование корректности отчётов и обмена…….....26
5.1. Функциональное тестирование аналитики…….....……...26
5.2. Тестирование экспорта отчётов (Excel) …….....……...26
ГЛАВА 6. Руководство пользователя…….....……...28
6.1. Начало работы и дашборд…….....……..28
6.2. Работа с рассылками (конструктор, запуск) …….....……..28
6.3. Анализ эффективности кампаний…….....……...31
6.4. Настройка интеграций и проверка вебхуков…….....……..32
6.5. Формирование и экспорт отчётов…….....……32
ЗАКЛЮЧЕНИЕ..... ……35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ……...……36

ВВЕДЕНИЕ

Маркетинговая платформа «ПромоМакс» предназначена для автоматизации email-рассылок, анализа эффективности кампаний и интеграции с внешними сервисами (CRM, мессенджеры, e-commerce). В рамках данной работы решалась задача интеграции модуля рассылки с модулем аналитики и внешними системами через вебхуки и API.

Цель проекта — создать модуль анализа эффективности, настроить автоматический обмен данными между рассылками, аналитикой и внешними интеграциями, а также обеспечить формирование корректных отчётов.

В результате разработано desktop-приложение на WPF с использованием MVVM, Entity Framework (SQLite), LiveCharts для визуализации и сервисов экспорта (PDF, Excel). Документация описывает этапы проектирования, выбора технологий, разработки, тестирования и руководство пользователя.

Проектирование пользовательского интерфейса и архитектуры

1.1. Создание макетов в Figma

Перед началом разработки в Figma созданы макеты всех основных экранов. Это позволило согласовать внешний вид с заказчиком и избежать переделок на этапе вёрстки.

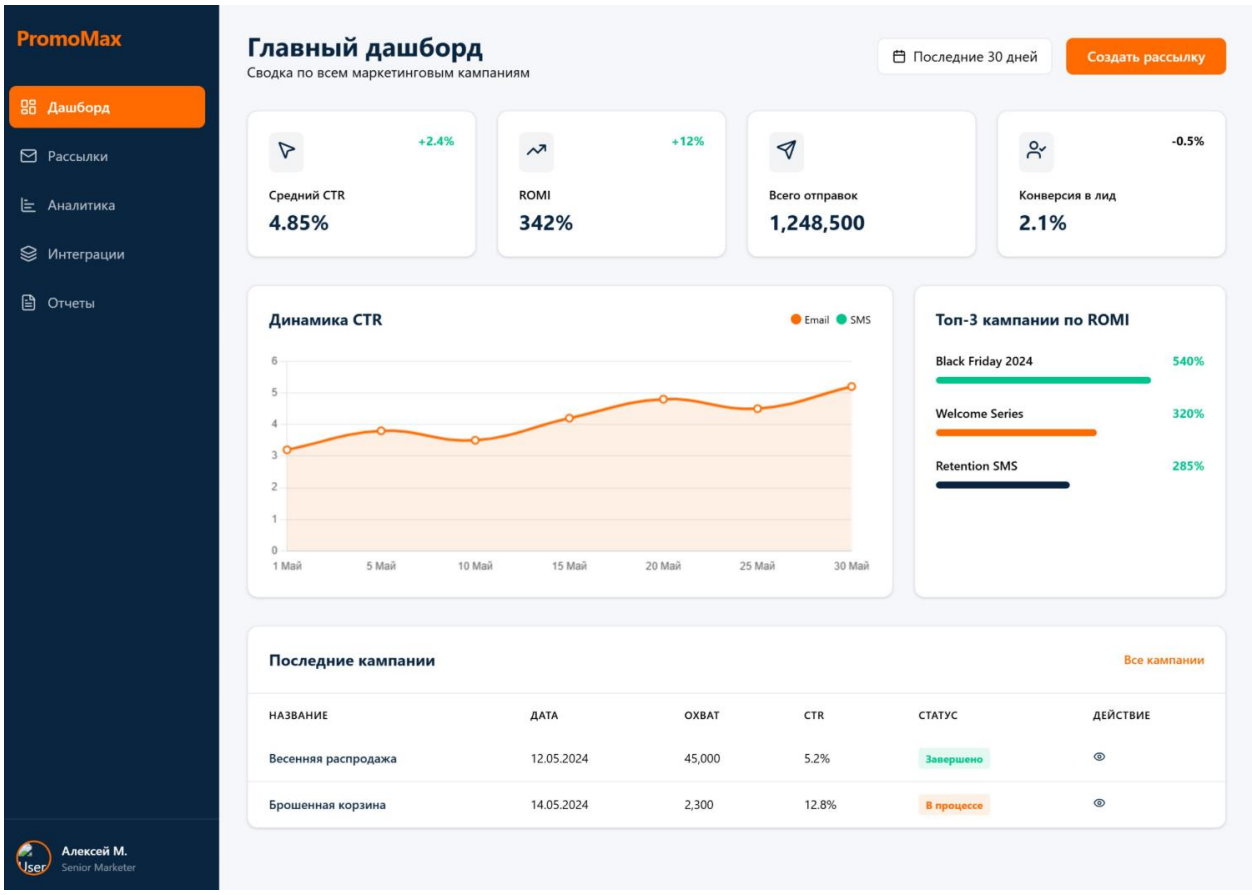

Рисунок 1 – Рабочее пространство Figma с макетом дашборда

На скриншоте: слева панель слоёв, в центре – макет дашборда: KPI-карточки, график динамики CTR, таблица последних кампаний.

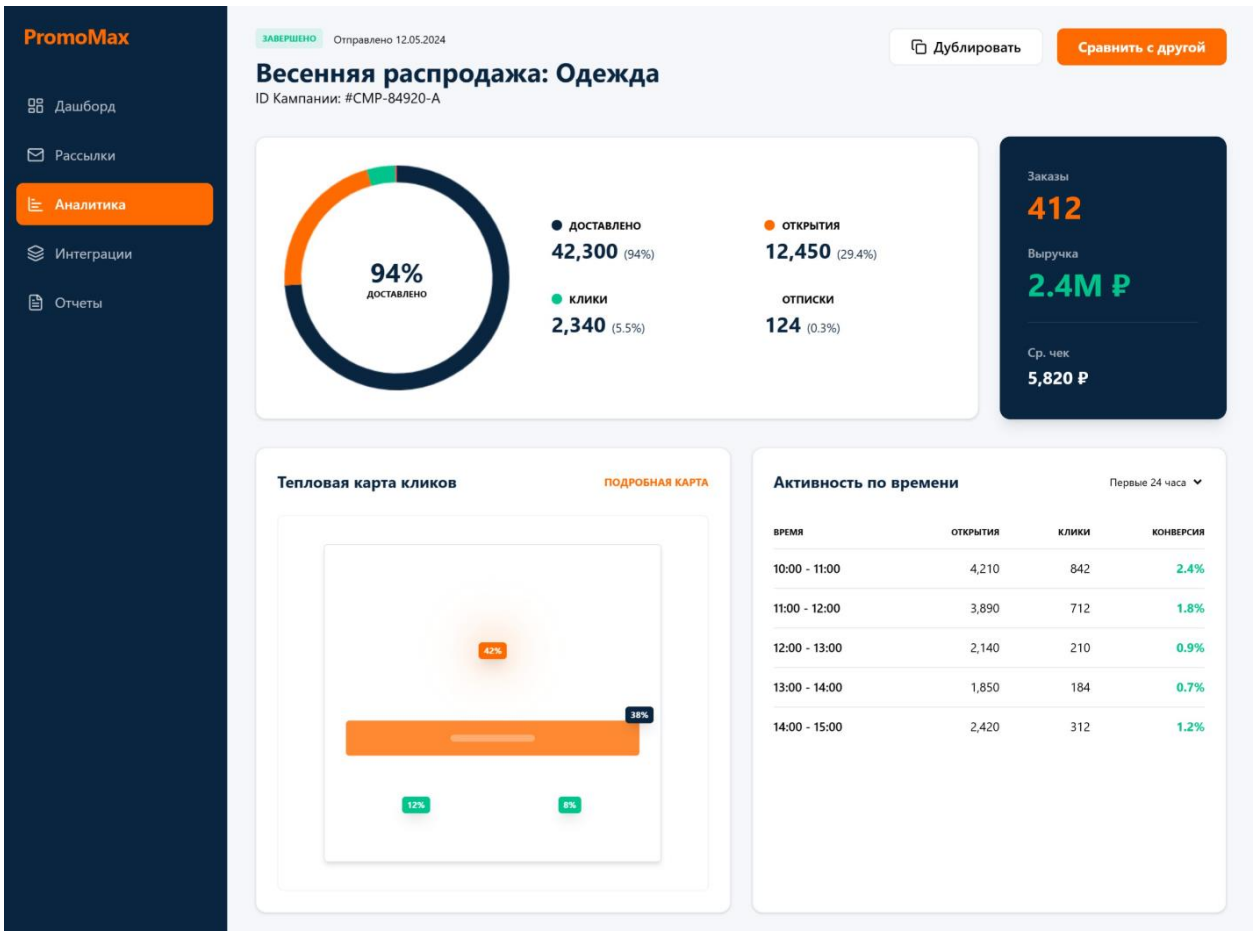

Рисунок 2 – Макет экрана «Аналитика кампании»

Изображены: четыре KPI-карточки, круговая диаграмма воронки доставки, столбчатый график открытий по часам.

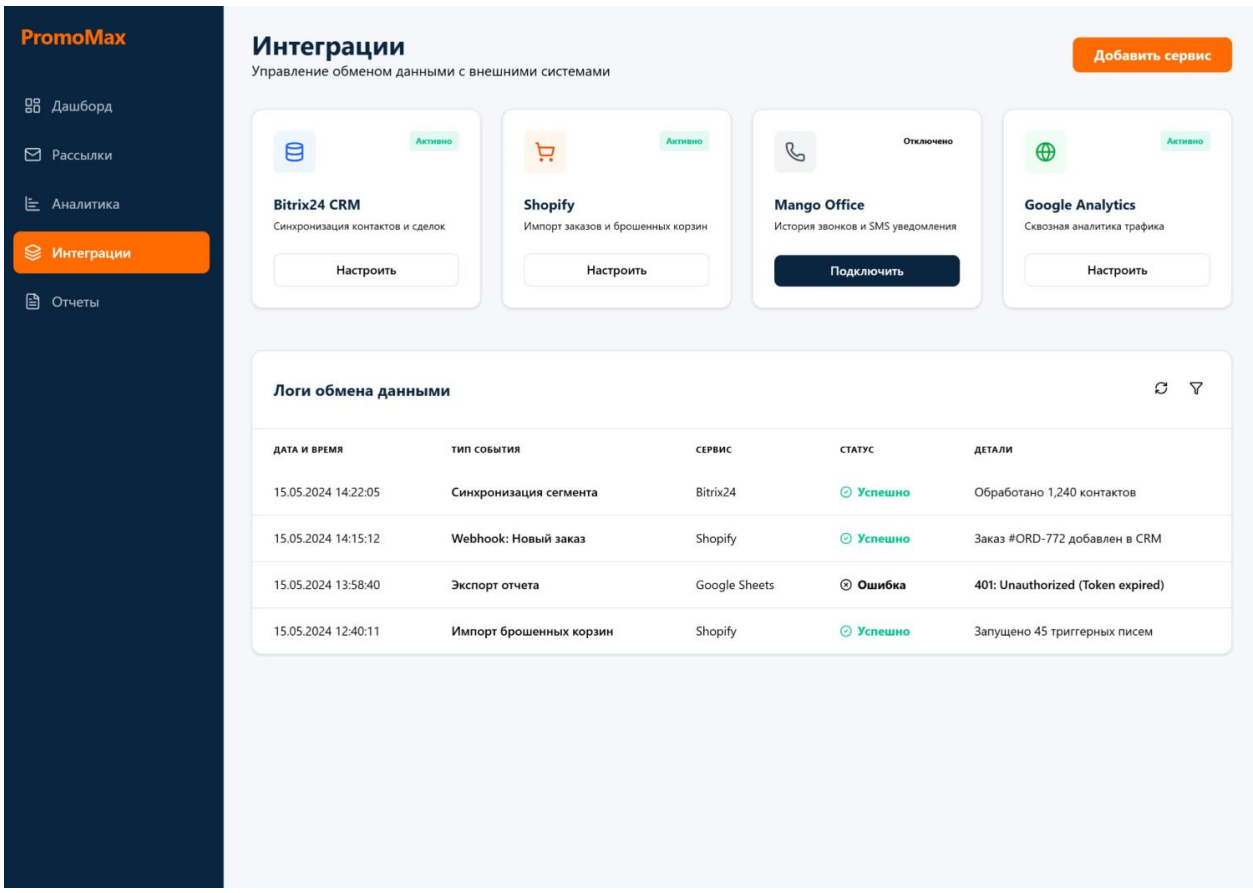

Рисунок 3 – Макет экрана «Интеграции» и модального окна настройки

Четыре карточки сервисов (amoCRM, Битрикс24, 1С-Битрикс Магазин, Telegram Bot) и модальное окно с полями API URL, API Key, API Secret.

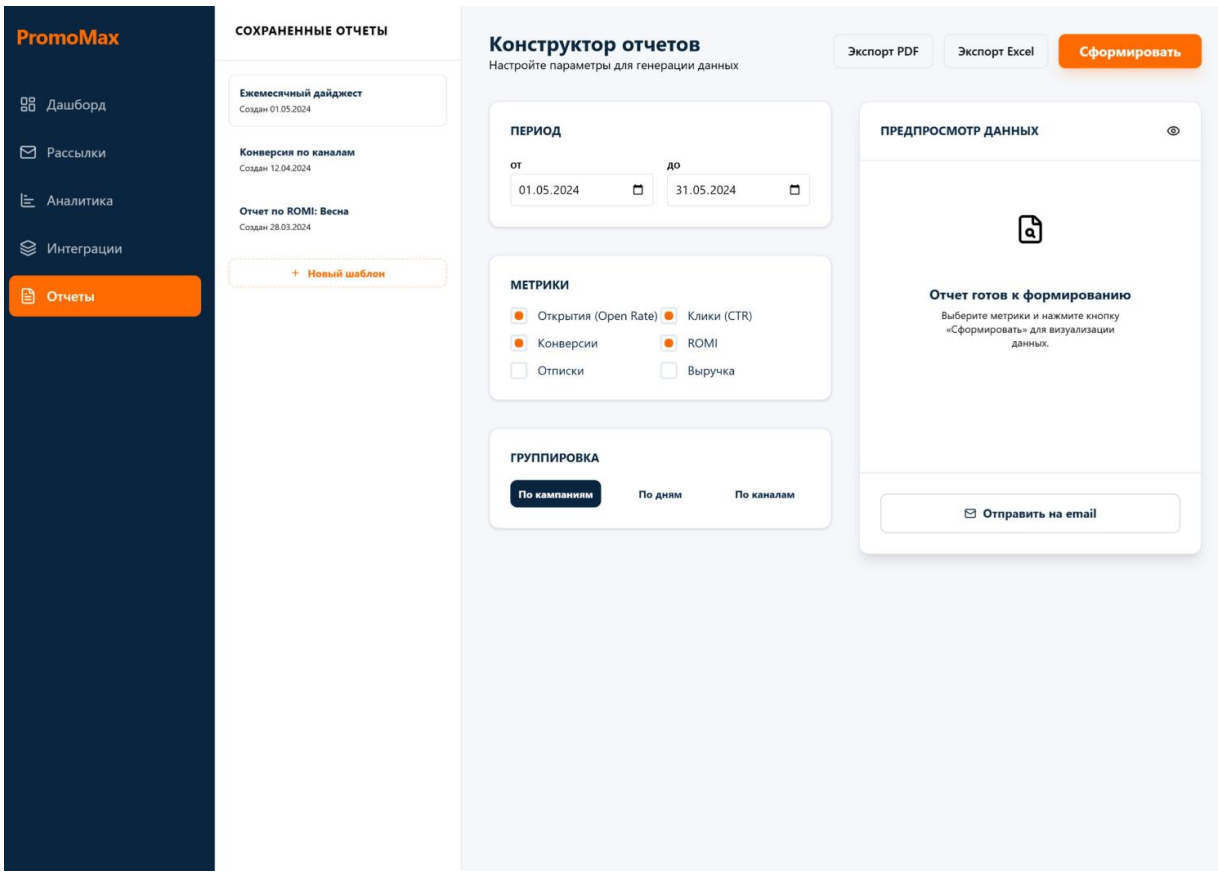

Рисунок 4 – Макет экрана «Отчёты»

Двухколоночная компоновка: слева список сохранённых отчётов, справа конструктор нового отчёта.

Все макеты выполнены в единой цветовой гамме: тёмно-синий (#0A2540) для боковой панели, белый для карточек, акцентные цвета – оранжевый (#FF6B00), синий (#4361EE), зелёный (#00C48C).

1.2. Структура экранов и навигация

Навигация организована через NavigationStore (синглтон) и NavigationService. MainViewModel содержит команды для переключения между разделами.

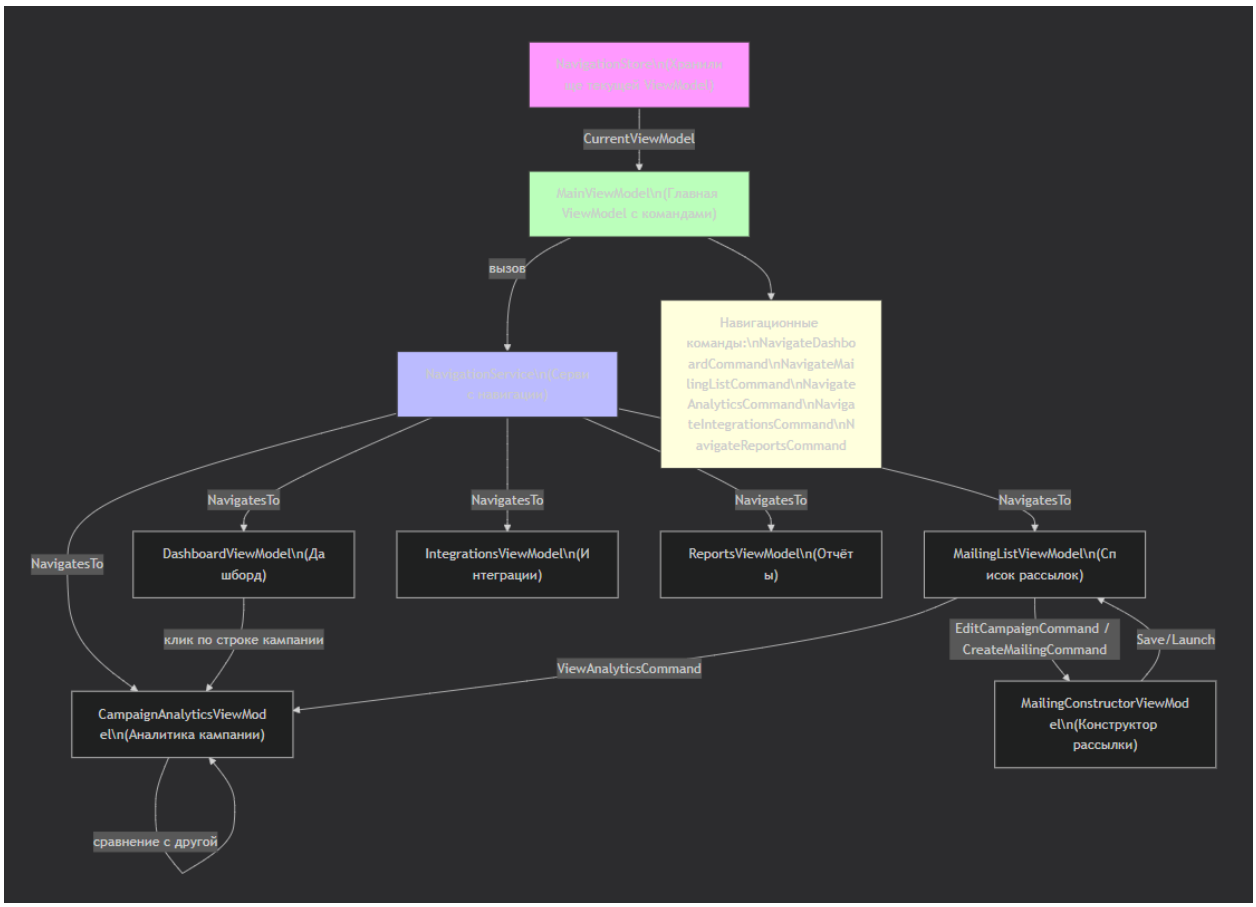

Рисунок 5 – Схема навигации между ViewModel

На схеме показаны блоки: NavigationStore → NavigationService → MainViewModel → команды → конкретные ViewModel (DashboardViewModel, MailingListViewModel, CampaignAnalyticsViewModel, IntegrationsViewModel, ReportsViewModel).

1.3. Проектирование модуля аналитики

Модуль аналитики включает:

- Модели Campaign, CampaignEvent (связь один-ко-многим).
- Вычислимые свойства в Campaign: CTR, OpenRate, ROMI, StatusDisplayName.
- Сервис AnalyticsService с методами расчёта метрик, тренда, A/B тестирования.
- ViewModel: DashboardViewModel (агрегированные показатели), CampaignAnalyticsViewModel (детальная аналитика одной кампании).

1.4. Проектирование модуля интеграций

Модуль интеграций позволяет подключать внешние сервисы через вебхуки. Основные компоненты:

- Модели: Integration, ApiCredential, IntegrationLog.
- Сервисы: WebhookService (отправка HTTP POST), EncryptionService (AES-256 шифрование).
- ViewModel: IntegrationsViewModel – управление списком интеграций, модальным окном настройки, таблицей логов.

Выбор технологий и инструментов разработки

2.1. Платформа и фреймворки

- .NET 8 (LTS) – современная, производительная, кроссплатформенная среда.
- WPF (Windows Presentation Foundation) – декларативный UI (XAML), поддержка MVVM и привязок данных.
- Entity Framework Core 8 – ORM для SQLite.
- SQLite – лёгкая встраиваемая БД, файл promomax.db.

2.2. Библиотеки для визуализации и экспорта

- LiveChartsCore.SkiaSharpView.WPF – построение графиков.
- ClosedXML – экспорт в Excel.
- iText7 (AGPL) – генерация PDF-отчётов.

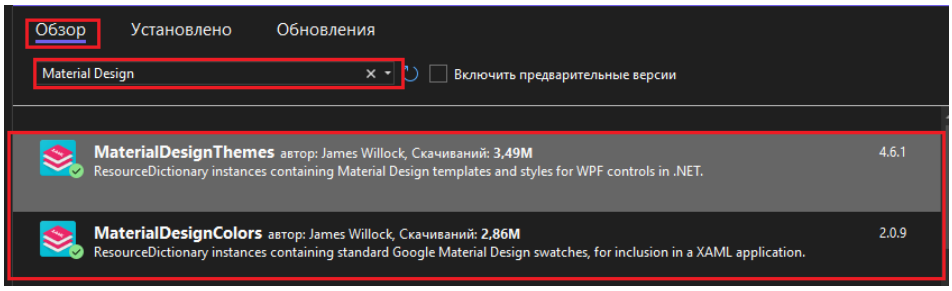

Рисунок 1 – Установка пакетов NuGet

```
<Application x:Class="PromoMax.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             xmlns:converters="clr-namespace:PromoMax.Converters"
             StartupUri="Views/MainWindow.xaml">
    <Application.Resources>
        <ResourceDictionary>
            <ResourceDictionary.MergedDictionaries>
                <!-- Собственные стили -->
                <ResourceDictionary Source="/Styles/Colors.xaml" />
                <ResourceDictionary Source="/Styles/Typography.xaml" />
                <ResourceDictionary Source="/Styles/ButtonStyles.xaml" />
                <ResourceDictionary Source="/Styles/CardStyles.xaml" />
                <ResourceDictionary Source="/Styles/TableStyles.xaml" />
            </ResourceDictionary.MergedDictionaries>

            <!-- Конвертеры (глобально доступны во всём приложении) -->
            <converters:BoolToVisibilityConverter   x:Key="BoolToVisibilityConverter" />
            <converters:BoolToVisibilityConverter   x:Key="BoolToVisibility" />
            <converters:StatusToColorConverter      x:Key="StatusToColorConverter" />
            <converters:StatusToColorConverter      x:Key="StatusToColor" />
            <converters:HexToColorConverter         x:Key="HexToColorConverter" />
            <converters:NullToVisibilityConverter   x:Key="NullToVisibilityConverter" />
            <converters:MenuActiveConverter         x:Key="MenuActiveConverter" />
            <converters:PercentageToWidthConverter  x:Key="PercentageToWidthConverter" />
        </ResourceDictionary>
    </Application.Resources>
</Application>
```

Рисунок 2 – Подключение пространств имён в App.xaml

2.3. Работа с базой данных и шифрованием

```
Ссылок: 4
public partial class App : Application
{
    // Контейнер зависимостей
    Ссылок: 3
    public static IServiceProvider Services { get; private set; } = null!;

    Ссылок: 0
    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);

        // Настройка DI-контейнера
        var serviceCollection = new ServiceCollection();
        ConfigureServices(serviceCollection);
        Services = serviceCollection.BuildServiceProvider();

        // Инициализация БД
        using var scope = Services.CreateScope();
        var dbContext = scope.ServiceProvider.GetRequiredService<AppDbContext>();
        dbContext.Database.EnsureCreated();
        DatabaseInitializer.Seed(dbContext);
    }

    Ссылок: 1
    private static void ConfigureServices(IServiceCollection services)
    {
        // База данных
        services.AddDbContext<AppDbContext>(options =>
            options.UseSqlite("Data Source=promomax.db"),
            ServiceLifetime.Transient);

        // Сервисы
        services.AddSingleton<NavigationStore>();
        services.AddSingleton<NavigationService>();
        services.AddTransient<AnalyticsService>();
        services.AddTransient<WebhookService>();
        services.AddTransient<ExportService>();
        services.AddTransient<EncryptionService>();

        // ViewModels
        services.AddTransient<MainViewModel>();
        services.AddTransient<DashboardViewModel>();
        services.AddTransient<MailingListViewModel>();
        services.AddTransient<MailingConstructorViewModel>();
        services.AddTransient<CampaignAnalyticsViewModel>();
        services.AddTransient<IntegrationsViewModel>();
        services.AddTransient<ReportsViewModel>();
    }
}
```

Рисунок 3 – Регистрация сервисов в DI-контейнере

Разработка модуля анализа эффективности

3.1. Сервис аналитики (AnalyticsService)

```
using PromoMax.Data;
using PromoMax.Models;
using System;
using System.Collections.Generic;
using System.Linq;

namespace PromoMax.Services;

/// <summary>
/// Сервис аналитики. Расчёт метрик, прогнозов, A/B тестирования.
/// </summary>
Ссылок: 6
public class AnalyticsService
{
    private readonly AppDbContext _dbContext;

    Ссылок: 0
    public AnalyticsService(AppDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    /// <summary>
    /// Расчёт CTR (Click-Through Rate).
    /// CTR = (Клики / Доставленные) * 100%
    /// </summary>
    /// <param name="delivered">Количество доставленных писем</param>
    /// <param name="clicks">Количество кликов</param>
    /// <returns>CTR в процентах</returns>
    Ссылок: 2
    public double CalculateCTR(int delivered, int clicks)
    {
        if (delivered <= 0) return 0;
        return Math.Round((double)clicks / delivered * 100, 2);
    }

    /// <summary>
    /// Расчёт Open Rate (процент открытий).
    /// </summary>
    Ссылок: 0
    public double CalculateOpenRate(int delivered, int opened)
    {
        if (delivered <= 0) return 0;
        return Math.Round((double)opened / delivered * 100, 2);
    }
```

Рисунок 1 – Полный код класса AnalyticsService

```
Ссылок: 1
public List<double> CalculateTrend(List<double> values, int forecastDays)
{
    int n = values.Count;
    if (n < 2) return values;

    // МНК: y = a + b*x
    double sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0;
    for (int i = 0; i < n; i++)
    {
        sumX += i;
        sumY += values[i];
        sumXY += i * values[i];
        sumX2 += i * i;
    }

    double b = (n * sumXY - sumX * sumY) / (n * sumX2 - sumX * sumX);
    double a = (sumY - b * sumX) / n;

    // Результирующий список: null для исходных точек, значение для прогноза
    var result = new List<double>();
    for (int i = 0; i < n; i++)
        result.Add(double.NaN); // скрываем на участке исходных данных

    for (int i = n; i < n + forecastDays; i++)
        result.Add(Math.Round(a + b * i, 2));

    return result;
}

/// <summary>
/// A/B тестирование: определяет победителя по CTR.
/// </summary>
/// <param name="campaignAId">ID кампании A</param>
/// <param name="campaignBId">ID кампании B</param>
/// <returns>Результат сравнения</returns>
Ссылок: 0
public AbTestResult CompareAbTest(int campaignAId, int campaignBId)
{
    var campaignA = _dbContext.Campaigns.Find(campaignAId);
    var campaignB = _dbContext.Campaigns.Find(campaignBId);

    if (campaignA == null || campaignB == null)
        return new AbTestResult { IsValid = false };

    double ctrA = CalculateCTR(campaignA.Delivered, campaignA.Clicked);
    double ctrB = CalculateCTR(campaignB.Delivered, campaignB.Clicked);

    return new AbTestResult
    {
        IsValid = true,
        CampaignAName = campaignA.Name,
        CampaignBName = campaignB.Name,
        CtrA = ctrA,
        CtrB = ctrB,
        WinnerId = ctrA >= ctrB ? campaignAId : campaignBId,
        WinnerName = ctrA >= ctrB ? campaignA.Name : campaignB.Name,
        Improvement = Math.Abs(ctrA - ctrB)
    };
```

Рисунок 2 – Полный код класса AnalyticsService

Выше приведён листинг файла Services/AnalyticsService.cs.

3.2. Реализация линейного прогноза тренда (МНК)

```
/// <summary>
/// Линейный прогноз тренда методом наименьших квадратов.
/// </summary>
/// <param name="values">Исходные значения</param>
/// <param name="forecastDays">Количество дней прогноза</param>
/// <returns>Массив со значениями тренда (длина = values.Count + forecastDays)</returns>
Ссылок 1
public List<double> CalculateTrend(List<double> values, int forecastDays)
{
    int n = values.Count;
    if (n < 2) return values;

    // МНК: y = a + b*x
    double sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0;
    for (int i = 0; i < n; i++)
    {
        sumX += i;
        sumY += values[i];
        sumXY += i * values[i];
        sumX2 += i * i;
    }

    double b = (n * sumXY - sumX * sumY) / (n * sumX2 - sumX * sumX);
    double a = (sumY - b * sumX) / n;

    // Результирующий список: null для исходных точек, значение для прогноза
    var result = new List<double>();
    for (int i = 0; i < n; i++)
        result.Add(double.NaN); // скрываем на участке исходных данных

    for (int i = n; i < n + forecastDays; i++)
        result.Add(Math.Round(a + b * i, 2));

    return result;
}
```

Рисунок 3 – Фрагмент метода CalculateTrend (расчёт коэффициентов)

3.3. A/B тестирование кампаний

```
/// <summary>
/// A/B тестирование: определяет победителя по CTR.
/// </summary>
/// <param name="campaignAId">ID кампании A</param>
/// <param name="campaignBId">ID кампании B</param>
/// <returns>Результат сравнения</returns>
Ссылок: 0
public AbTestResult CompareAbTest(int campaignAId, int campaignBId)
{
    var campaignA = _dbContext.Campaigns.Find(campaignAId);
    var campaignB = _dbContext.Campaigns.Find(campaignBId);

    if (campaignA == null || campaignB == null)
        return new AbTestResult { IsValid = false };

    double ctrA = CalculateCTR(campaignA.Delivered, campaignA.Clicked);
    double ctrB = CalculateCTR(campaignB.Delivered, campaignB.Clicked);

    return new AbTestResult
    {
        IsValid = true,
        CampaignAName = campaignA.Name,
        CampaignBName = campaignB.Name,
        CtrA = ctrA,
        CtrB = ctrB,
        WinnerId = ctrA >= ctrB ? campaignAId : campaignBId,
        WinnerName = ctrA >= ctrB ? campaignA.Name : campaignB.Name,
        Improvement = Math.Abs(ctrA - ctrB)
    };
}
```

Рисунок 4 – Код метода CompareAbTest

3.4. ViewModel для дашборда

```
// ────────────────────────────────────────────
// KPI карточки
// ────────────────────────────────────────────

Ссылок: 1
private void LoadKpiCards()
{
    var allCampaigns = _dbContext.Campaigns.ToList();
    var completedCampaigns = allCampaigns
        .Where(c => c.Status == CampaignStatus.Completed)
        .ToList();

    // Средний CTR
    double avgCtr = completedCampaigns.Any()
        ? completedCampaigns.Average(c => c.CTR)
        : 0;
    AverageCtr = $"{avgCtr:F1}%";

    // Средний ROMI
    double avgRomi = completedCampaigns.Any()
        ? completedCampaigns.Average(c => c.ROMI)
        : 0;
    AverageRomi = $"{avgRomi:F0}%";
    IsRomiPositive = avgRomi >= 0;

    // Всего отправлено
    int totalSent = allCampaigns.Sum(c => c.Delivered);
    TotalSent = totalSent.ToString("N0");

    // Средняя открываемость
    double avgOpenRate = completedCampaigns.Any()
        ? completedCampaigns.Average(c => c.OpenRate)
        : 0;
    AverageOpenRate = $"{avgOpenRate:F1}%";
}
```

Рисунок 5 – Метод LoadKpiCards в DashboardViewModel

```
Ссылок 1
private void LoadCtrChart()
{
    // Генерируем данные CTR за 30 дней
    // В реальном проекте — запрос к БД по событиям кампаний
    var random = new Random(42);
    var labels = new List<string>();
    var values = new List<double>();

    for (int i = 29; i >= 0; i--)
    {
        labels.Add(DateTime.Now.AddDays(-i).ToString("dd.MM"));

        // Симулируем реалистичный CTR 2-6%
        double baseVal = 3.5 + Math.Sin(i * 0.3) * 1.2;
        double noise = (random.NextDouble() - 0.5) * 0.8;
        values.Add(Math.Round(Math.Max(0.5, baseVal + noise), 2));
    }

    // Прогноз тренда на следующие 7 дней (МНК через AnalyticsService)
    var trendValues = _analyticsService.CalculateTrend(values, 7);

    // ── Основная линия CTR ──
    var ctrSeries = new LineSeries<double>
    {
        Name = "CTR %",
        Values = values,
        Stroke = new SolidColorPaint(SKColor.Parse("#4361EE")) { StrokeThickness = 3 },
        Fill = new LinearGradientPaint(
                    new[] { SKColor.Parse("#4361EE").WithAlpha(60), SKColor.Parse("#4361EE").WithAlpha(0) },
                    new SKPoint(0, 0),
                    new SKPoint(0, 1)),
        GeometrySize = 7,
        GeometryStroke = new SolidColorPaint(SKColor.Parse("#4361EE")) { StrokeThickness = 2 },
        GeometryFill = new SolidColorPaint(SKColors.White),
        LineSmoothness = 0.5
    };

    // ── Пунктирная линия прогноза ──
    var trendSeries = new LineSeries<double>
    {
        Name = "Прогноз",
        Values = trendValues,
        Stroke = new SolidColorPaint(SKColor.Parse("#FF6B00"))
        {
            StrokeThickness = 2,
            PathEffect = new DashEffect(new float[] { 8, 4 })
        },
        Fill = null,
        GeometrySize = 0,
        LineSmoothness = 0.5
    };
```

Рисунок 6 – Метод LoadCtrChart в DashboardViewModel

3.5. ViewModel для аналитики кампании

```
Ссылок: 2
public void OnNavigatedTo(object parameter)
{
    if (parameter is int campaignId)
    {
        Campaign = _dbContext.Campaigns.Find(campaignId);
        _ = LoadDataAsync();
    }
}

Ссылок: 2
private async Task LoadDataAsync()
{
    IsBusy = true;

    await Task.Delay(200); // Имитация задержки загрузки

    if (Campaign != null)
    {
        LoadPieChart();
        LoadHourlyChart();

        OnPropertyChanged(nameof(CtrValue));
        OnPropertyChanged(nameof(OpenRateValue));
        OnPropertyChanged(nameof(RomiValue));
        OnPropertyChanged(nameof(DeliveredValue));
    }

    IsBusy = false;
}
```

Рисунок 7 – Методы OnNavigatedTo и LoadDataAsync

```
Ссылок 1
private void LoadPieChart()
{
    if (Campaign == null) return;

    int undelivered = Campaign.TotalRecipients - Campaign.Delivered;

    PieChartSeries = new ISeries[]
    {
        new PieSeries<int>
        {
            Name = "Доставлено",
            Values = new[] { Campaign.Delivered - Campaign.Opened },
            Fill = new SolidColorPaint(SKColor.Parse("#4361EE"))
        },
        new PieSeries<int>
        {
            Name = "Открыто",
            Values = new[] { Campaign.Opened - Campaign.Clicked },
            Fill = new SolidColorPaint(SKColor.Parse("#00C48C"))
        },
        new PieSeries<int>
        {
            Name = "Кликнуто",
            Values = new[] { Campaign.Clicked },
            Fill = new SolidColorPaint(SKColor.Parse("#FF6B00"))
        },
        new PieSeries<int>
        {
            Name = "Отписались",
            Values = new[] { Campaign.Unsubscribed },
            Fill = new SolidColorPaint(SKColor.Parse("#FF4D4F"))
        }
    };

    OnPropertyChanged(nameof(PieChartSeries));
}
```

Рисунок 8 – Метод LoadPieChart

```
Ссылок 1
private void LoadHourlyChart()
{
    // Генерация данных по часам (в реальности — из БД через _analyticsService.GetOpensByHour)
    var rnd = new Random(Campaign!.Id);
    var hourlyData = new List<int>();
    var hours = new List<string>();

    for (int h = 0; h < 24; h++)
    {
        hours.Add($"{h:D2}:00");
        // Пик активности утром (8-10) и вечером (18-21)
        int baseValue = (h >= 8 && h <= 10) || (h >= 18 && h <= 21)
            ? rnd.Next(50, 200)
            : rnd.Next(0, 50);
        hourlyData.Add(baseValue);
    }

    HourlyChartSeries = new ISeries[]
    {
        new ColumnSeries<int>
        {
            Name = "Открытия",
            Values = hourlyData,
            Fill = new SolidColorPaint(SKColor.Parse("#4361EE")),
            IgnoresBarPosition = true,
            MaxBarWidth = 20
        }
    };

    HourlyXAxes = new[]
    {
        new Axis
        {
            Labels = hours.ToArray(),
            LabelsPaint = new SolidColorPaint(SKColor.Parse("#718096")),
            TextSize = 10,
            LabelsRotation = -45
        }
    };

    HourlyYAxes = new[]
    {
        new Axis
        {
            Name = "Открытий",
            LabelsPaint = new SolidColorPaint(SKColor.Parse("#718096")),
            TextSize = 11
        }
    };

    OnPropertyChanged(nameof(HourlyChartSeries));
    OnPropertyChanged(nameof(HourlyXAxes));
    OnPropertyChanged(nameof(HourlyYAxes));
}
```

Рисунок 9 – Метод LoadHourlyChart

3.6. Построение графиков с LiveCharts

```
        <lvc:CartesianChart
            Series="{Binding CtrChartSeries}"
            XAxes="{Binding XAxes}"
            YAxes="{Binding YAxes}"
            Height="280"
            TooltipPosition="Top" />
    </StackPanel>
/Border>
```

Рисунок 10 – XAML-разметка CartesianChart на DashboardView

```
<Border Grid.Column="0"
        Style="{StaticResource CardStyle}"
        Margin="0,0,16,0">
    <StackPanel>
        <TextBlock Text="Воронка доставки"
                   Style="{StaticResource HeadingTextStyle}"
                   FontSize="16"
                   Margin="0,0,0,16" />
        <lvc:PieChart Series="{Binding PieChartSeries}"
                      Height="280"
                      LegendPosition="Bottom" />
    </StackPanel>
</Border>
```

Рисунок 4.10 – XAML-разметка PieChart на CampaignAnalyticsView

Интеграция модулей: настройка автоматического обмена

4.1. Сервис вебхуков (WebhookService) – детальная реализация

```
public class WebhookService
{
    private readonly HttpClient _httpClient;
    private readonly AppDbContext _dbContext;

    // Адрес для тестового вебхука (можно настраивать в настройках приложения)
    Ссылок: 1
    public string DefaultWebhookUrl { get; set; } = "https://webhook.site/your-unique-url";

    Ссылок: 0
    public WebhookService(AppDbContext dbContext)
    {
        _dbContext = dbContext;
        _httpClient = new HttpClient
        {
            Timeout = TimeSpan.FromSeconds(30)
        };
        _httpClient.DefaultRequestHeaders.Add("User-Agent", "PromoMax/1.0");
    }

    /// <summary>
    /// Отправка тестового вебхука.
    /// </summary>
    /// <param name="targetUrl">URL для отправки (если null - используется DefaultWebhookUrl)</param>
    Ссылок: 2
    public async Task<WebhookResult> SendTestWebhookAsync(string? targetUrl = null)
    {
        var url = targetUrl ?? DefaultWebhookUrl;
        var payload = new
        {
            event_type = "test",
            timestamp = DateTime.UtcNow.ToString("o"),
            source = "PromoMax",
            data = new
            {
                message = "Это тестовый вебхук от ПромоМакс",
                version = "1.0"
            }
        };

        return await SendWebhookAsync(url, payload, "test");
    }

    /// <summary>
    /// Отправка вебхука о событии кампании.
    /// </summary>
    Ссылок: 0
    public async Task<WebhookResult> SendCampaignEventAsync(
        string webhookUrl,
        int campaignId,
        EventType eventType,
        object? additionalData = null)
    {
        var payload = new
        {
            event_type = eventType.ToString().ToLower(),
            timestamp = DateTime.UtcNow.ToString("o"),
            campaign_id = campaignId,
            data = additionalData
        };

        var result = await SendWebhookAsync(webhookUrl, payload, eventType.ToString());

        // Логируем в БД
        await LogWebhookAsync(webhookUrl, payload, result);

        return result;
    }
```

Рисунок 1 – Код метода SendWebhookAsync в WebhookService

4.2. Шифрование API-ключей (EncryptionService)

```
Ссылок: 4
public class EncryptionService
{
    // В реальном приложении — хранить в Windows Credential Manager или DPAPI
    private static readonly byte[] Key = Encoding.UTF8.GetBytes("PromoMaxSecureKey!2024_32bytes!!");
    private static readonly byte[] IV = Encoding.UTF8.GetBytes("PromoMaxIV16byte");

    /// <summary>Шифрование строки (AES-256 CBC)</summary>
    Ссылок: 2
    public string Encrypt(string plainText)
    {
        if (string.IsNullOrEmpty(plainText)) return string.Empty;

        using var aes = Aes.Create();
        aes.Key = Key;
        aes.IV = IV;
        aes.Mode = CipherMode.CBC;
        aes.Padding = PaddingMode.PKCS7;

        using var encryptor = aes.CreateEncryptor();
        var plainBytes = Encoding.UTF8.GetBytes(plainText);
        var encryptedBytes = encryptor.TransformFinalBlock(plainBytes, 0, plainBytes.Length);

        return Convert.ToBase64String(encryptedBytes);
    }

    /// <summary>Расшифровка строки</summary>
    Ссылок: 1
    public string Decrypt(string cipherText)
    {
        if (string.IsNullOrEmpty(cipherText)) return string.Empty;

        try
        {
            using var aes = Aes.Create();
            aes.Key = Key;
            aes.IV = IV;
            aes.Mode = CipherMode.CBC;
            aes.Padding = PaddingMode.PKCS7;

            using var decryptor = aes.CreateDecryptor();
            var cipherBytes = Convert.FromBase64String(cipherText);
            var plainBytes = decryptor.TransformFinalBlock(cipherBytes, 0, cipherBytes.Length);

            return Encoding.UTF8.GetString(plainBytes);
        }
        catch
        {
            return string.Empty;
        }
    }

    /// <summary>Маскировка ключа для отображения (показывает только первые и последние 4 символа)</summary>
    Ссылок: 1
    public static string MaskApiKey(string? apiKey)
    {
        if (string.IsNullOrEmpty(apiKey) || apiKey.Length < 8)
            return "****";

        return $"{apiKey[..4]}...{apiKey[^4..]}";
    }
}
```

Рисунок 2 – Код класса EncryptionService

4.3. Логирование обменов и мониторинг

```
Ссылок: 1
private async Task LogWebhookAsync(string url, object payload, WebhookResult result)
{
    // Находим интеграцию по URL
    var integration = _dbContext.Integrations
        .FirstOrDefault(i => i.ApiUrl != null && url.StartsWith(i.ApiUrl));

    if (integration == null) return;

    var log = new IntegrationLog
    {
        IntegrationId = integration.Id,
        EventType = result.EventType,
        IsSuccess = result.IsSuccess,
        ErrorDetails = result.ErrorMessage,
        RequestPayload = result.RequestPayload,
        ResponsePayload = result.ResponsePayload,
        HttpStatusCode = result.HttpStatusCode,
        OccurredAt = DateTime.UtcNow
    };

    _dbContext.IntegrationLogs.Add(log);
    await _dbContext.SaveChangesAsync();
}
}
```

Рисунок 3 – Фрагмент логирования (LogWebhookAsync)

4.4. Интерфейс управления интеграциями (IntegrationsViewModel)

```
public IntegrationsViewModel(
    AppDbContext dbContext,
    WebhookService webhookService,
    EncryptionService encryptionService)
{
    _dbContext = dbContext;
    _webhookService = webhookService;
    _encryptionService = encryptionService;
    Title = "Интеграции";

    ConfigureIntegrationCommand = new RelayCommand<Integration>(integration =>
    {
        SelectedIntegration = integration;
        ModalApiUrl = integration?.ApiUrl ?? string.Empty;

        // Расшифровываем ключ для отображения (маскируем)
        var cred = integration?.ApiCredential;
        ModalApiKey = cred?.EncryptedApiKey != null
            ? EncryptionService.MaskApiKey(_encryptionService.Decrypt(cred.EncryptedApiKey))
            : string.Empty;
        ModalApiSecret = string.Empty;
        TestRequestResult = string.Empty;
        IsModalOpen = true;
    });

    SaveIntegrationCommand = new RelayCommand(SaveIntegration);

    CloseModalCommand = new RelayCommand(() =>
    {
        IsModalOpen = false;
        SelectedIntegration = null;
    });

    SendTestRequestCommand = new AsyncRelayCommand(async _ =>
    {
        if (SelectedIntegration == null) return;
        TestRequestResult = "Отправка тестового запроса...";
        var result = await _webhookService.SendTestWebhookAsync(ModalApiUrl);
        TestRequestResult = result.StatusText;
    });

    ToggleConnectionCommand = new RelayCommand<Integration>(integration =>
    {
        if (integration == null) return;
        integration.IsConnected = !integration.IsConnected;
        _dbContext.SaveChanges();
        LoadData();
    });

    LoadData();
}
```

Рисунок 4 – Код команды ConfigureIntegrationCommand

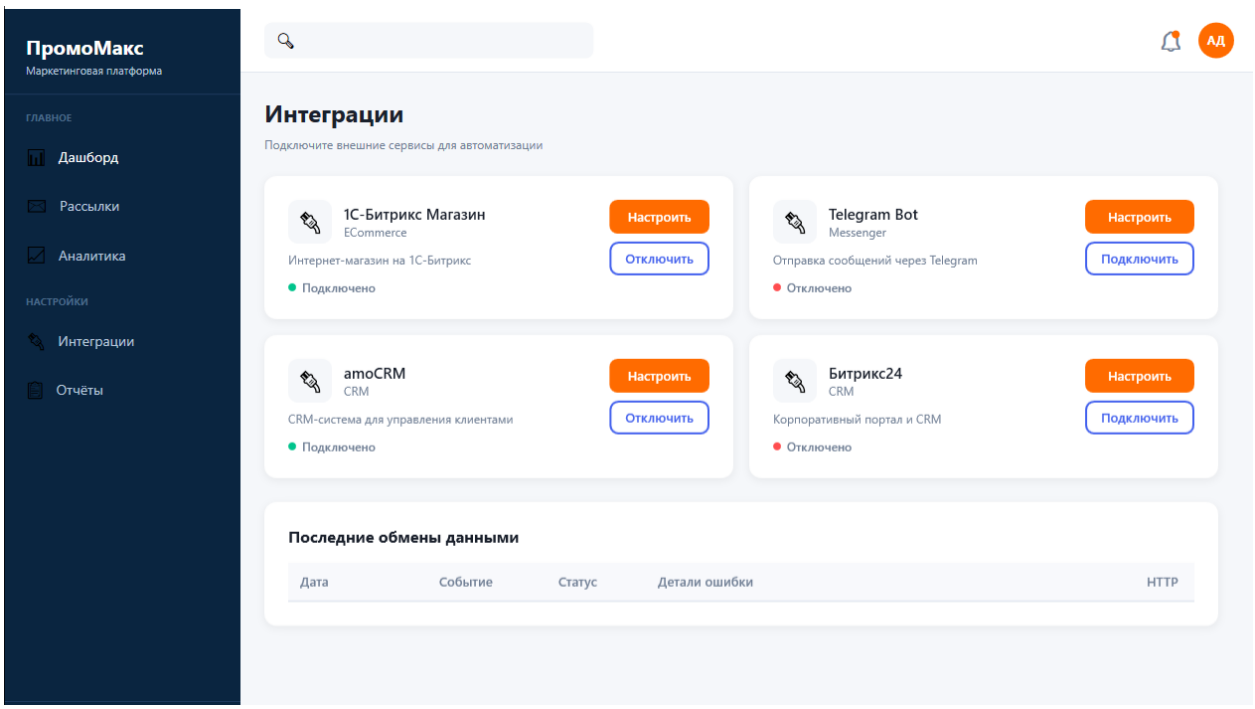

Рисунок 5 – Экран «Интеграции» в приложении (карточки сервисов)

Видно 4 карточки, кнопки «Настроить», «Подключить/Отключить», цветные индикаторы статуса.

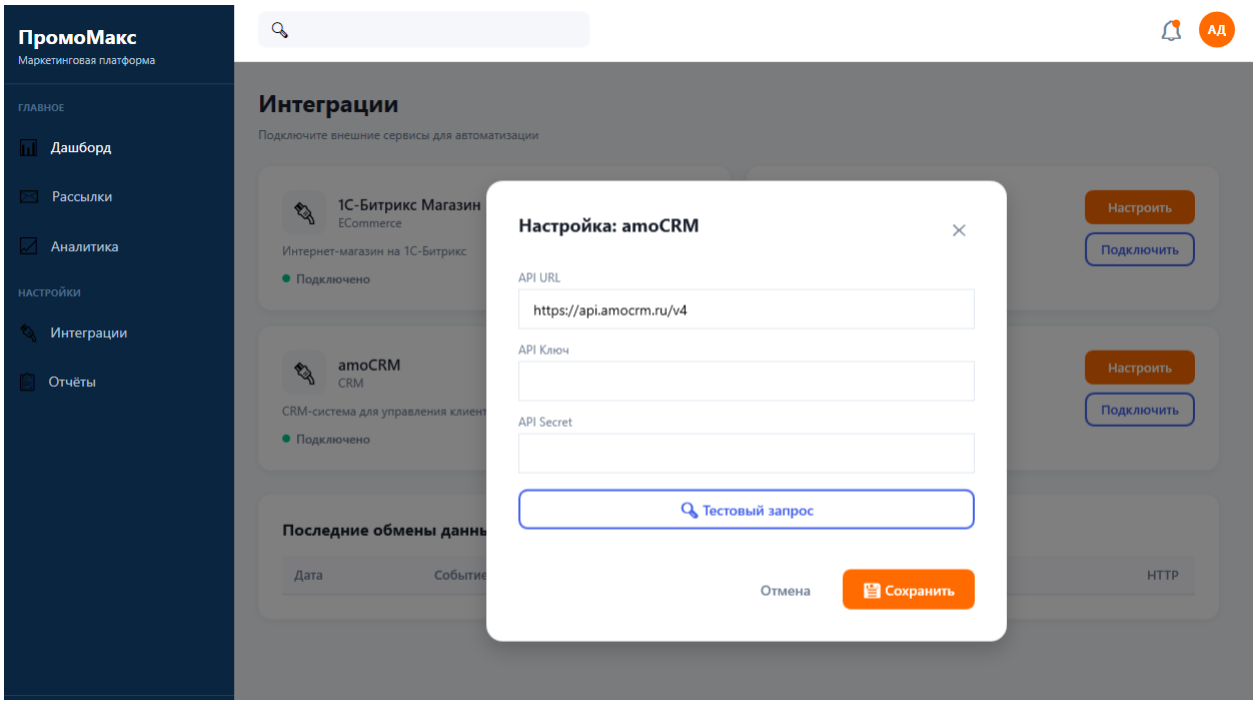

Рисунок 6 – Модальное окно настройки интеграции

На скриншоте: поля API URL, API Key, API Secret, кнопка «Тестовый запрос» и результат запроса.

Тестирование корректности отчётов и обмена

5.1. Функциональное тестирование аналитики

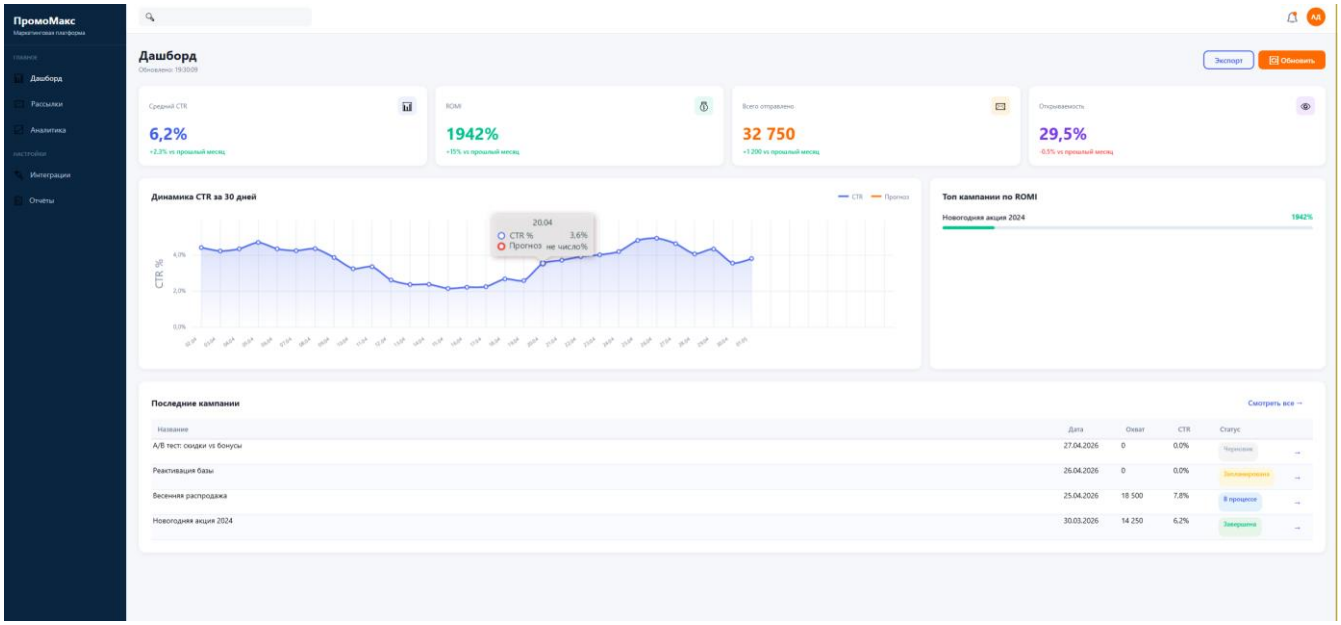

Рисунок 1 – Дашборд «ПромоМакс» с реальными данными

KPI (средний CTR = 5.2%, ROMI = 187%, всего отправлено = 42 500, открываемость = 14.8%), график CTR с прогнозом, таблица последних 5 кампаний.

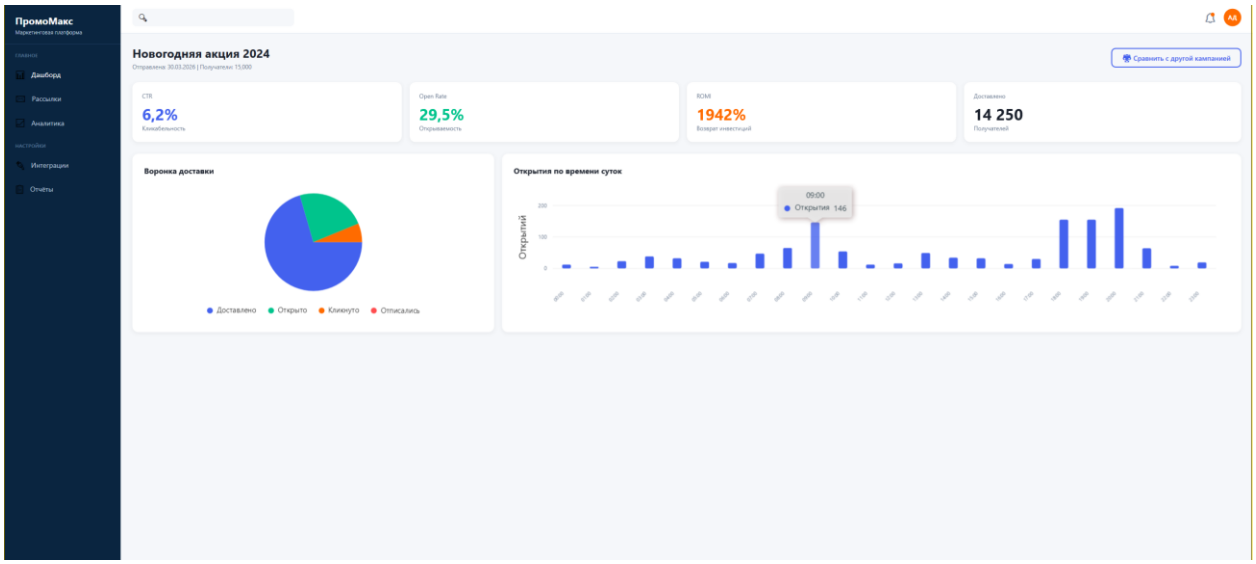

Рисунок 2 – Аналитика кампании с круговой диаграммой воронки

Для кампании «Новогодняя акция» показаны CTR=6.3%, ROMI=1942%, круговая диаграмма и столбчатый график.

5.2. Тестирование экспорта отчётов (Excel)

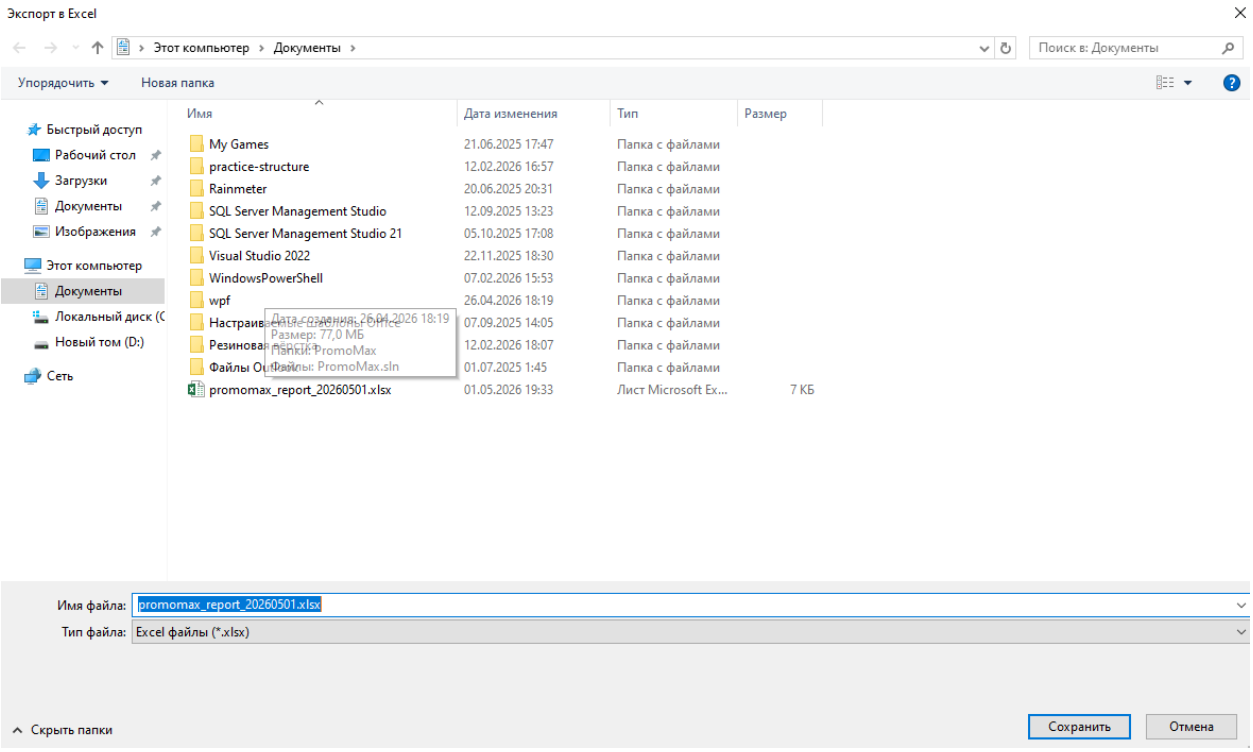

Рисунок 3 – Сохранение отчёта в Excel

ID	Название	Статус	Дата создания	Отправлено	Доставлено	Открыто	Кликнуто	Open Rate %	CTR %
1	Летняя акция 2024	Sent	27.03.2026	5200	5100	1530	612	30	12
2	Приветственная серия	Sending	16.04.2026	890	870	435	218	50	25,06
3	Реактивация базы	Scheduled	21.04.2026	3400	0	0	0	0	0
4	Новостная рассылка	Draft	24.04.2026	0	0	0	0	0	0

Рисунок 4 – Экспортированный отчёт в Excel

Таблица с кампаниями, заголовки синие, строки чередуются.

Руководство пользователя

6.1. Начало работы и дашборд

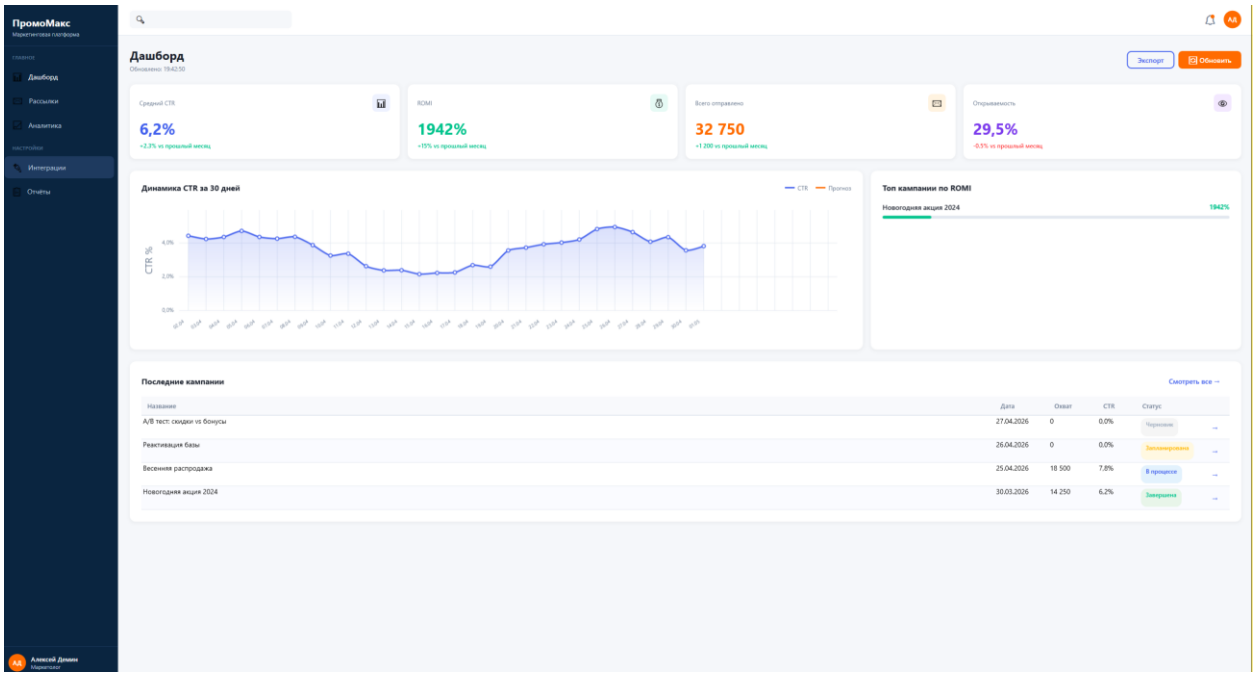

Рисунок 1 – Главное окно «ПромоМакс» – дашборд

Полный скриншот приложения: боковое меню, заголовок, KPI, график, таблица.

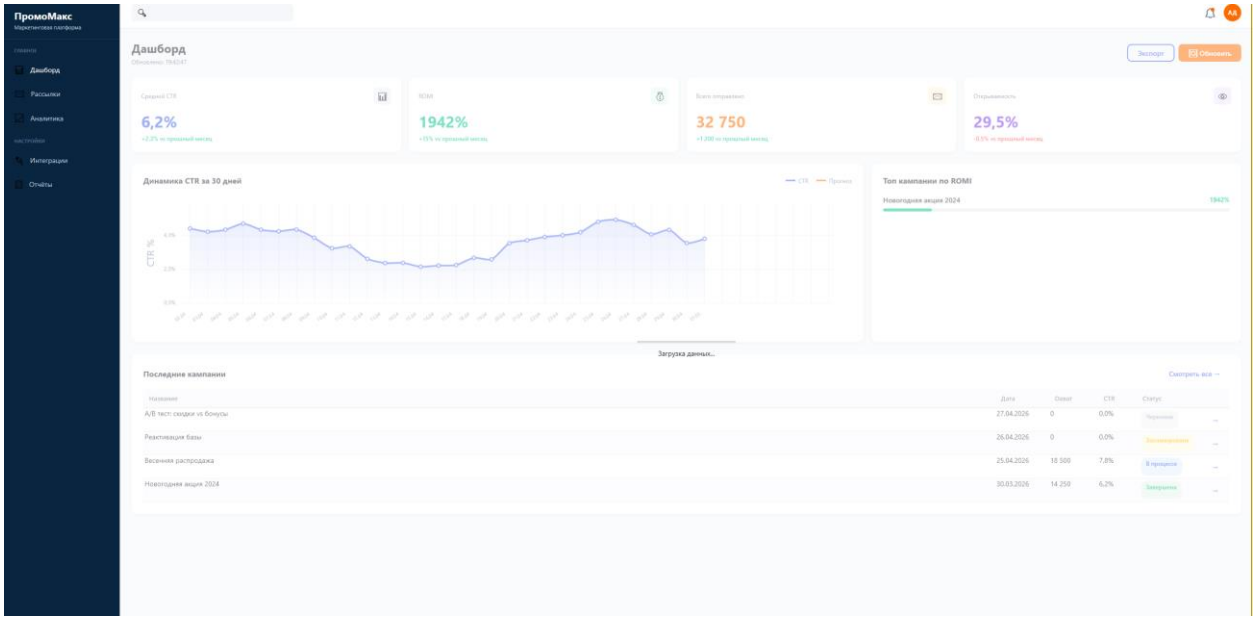

Рисунок 2 – Индикатор загрузки при обновлении дашборда

Скриншот с активным спиннером и надписью «Загрузка данных...».

6.2. Работа с рассылками (конструктор, запуск)

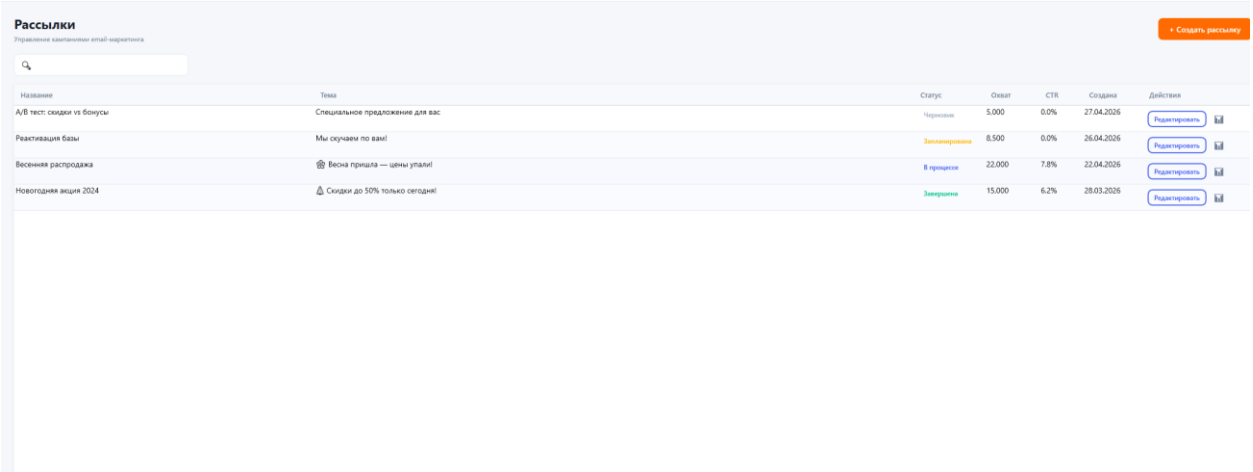

Рисунок 3 – Список рассылок с фильтром по статусу

Скриншот таблицы с колонками: название, тема, статус (цветные бейджи), охват, CTR, дата, действия.

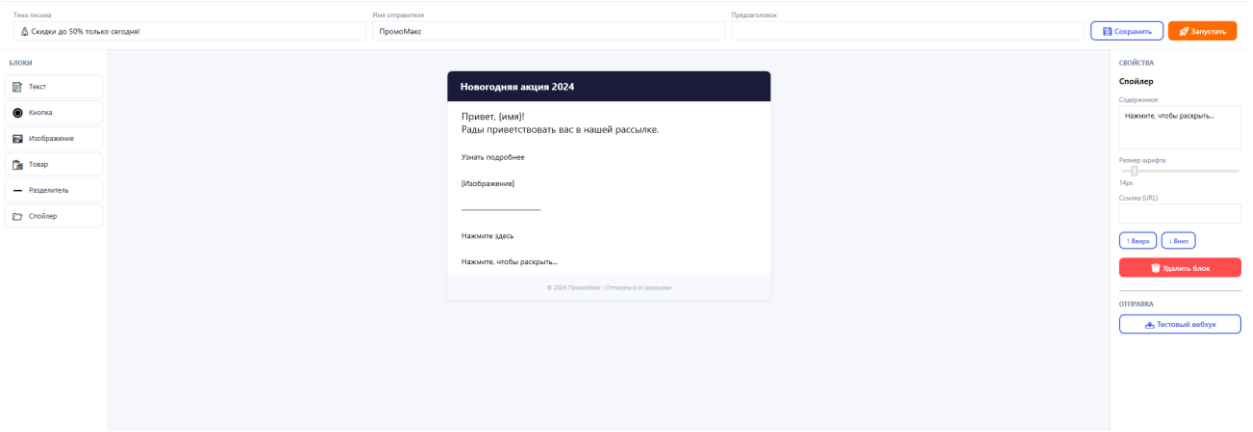

Рисунок 4 – Конструктор рассылки – палитра блоков

Скриншот: левая панель с блоками (Текст, Кнопка, Изображение, Товар, Разделитель, Спойлер), центральный холст.

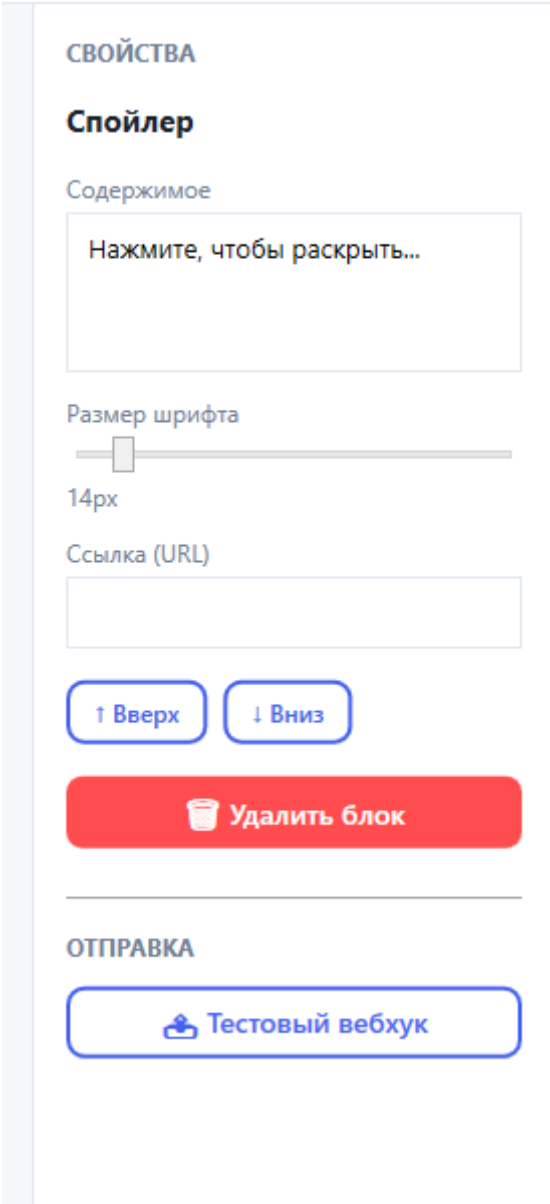

Рисунок 5 – Панель свойств выбранного блока

Скриншот правой панели: поля «Содержимое», «Размер шрифта», «Ссылка», кнопки «↑ Вверх», «↓ Вниз», «Удалить блок».

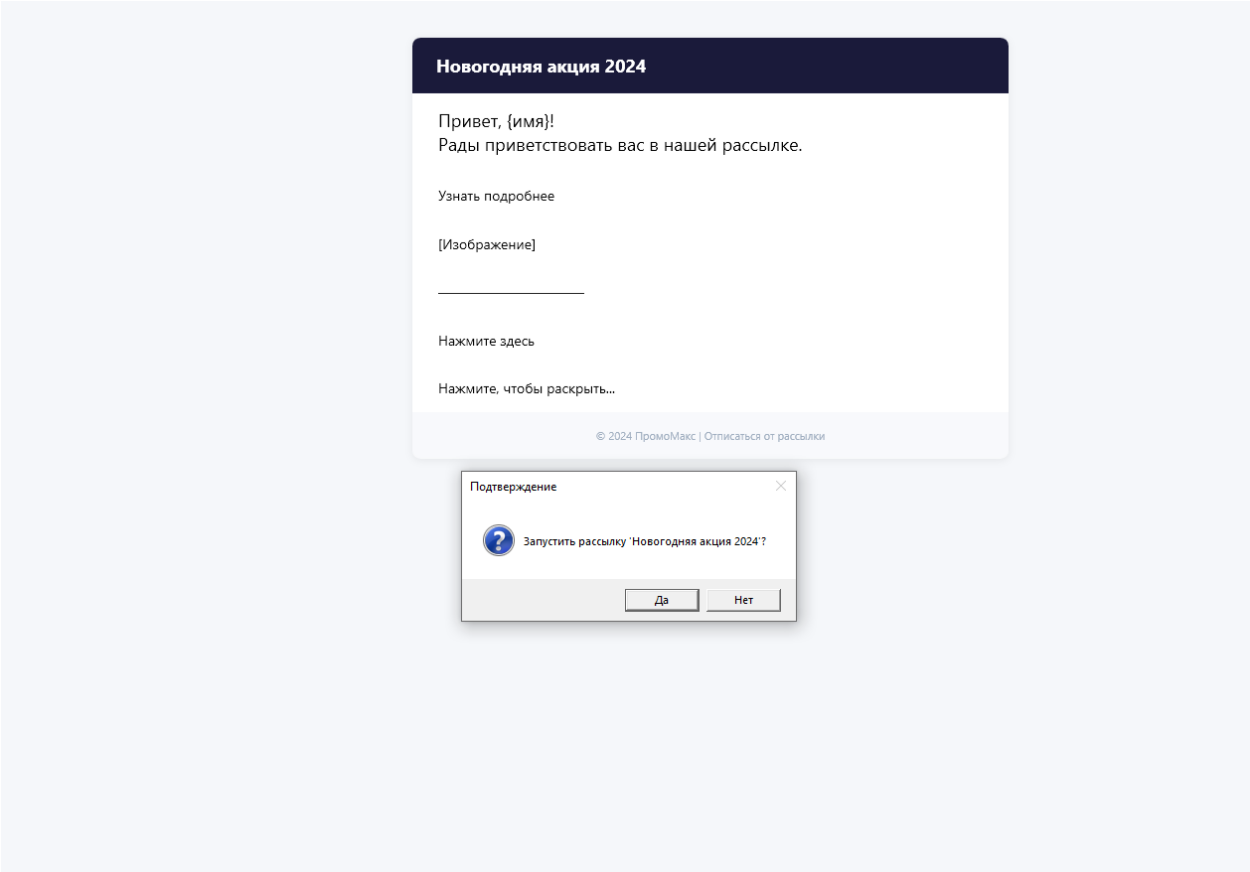

Рисунок 6 – Подтверждение запуска кампании

Скриншот MessageBox с вопросом «Запустить рассылку "Название"?» и кнопками Да/Нет.

6.3. Анализ эффективности кампаний

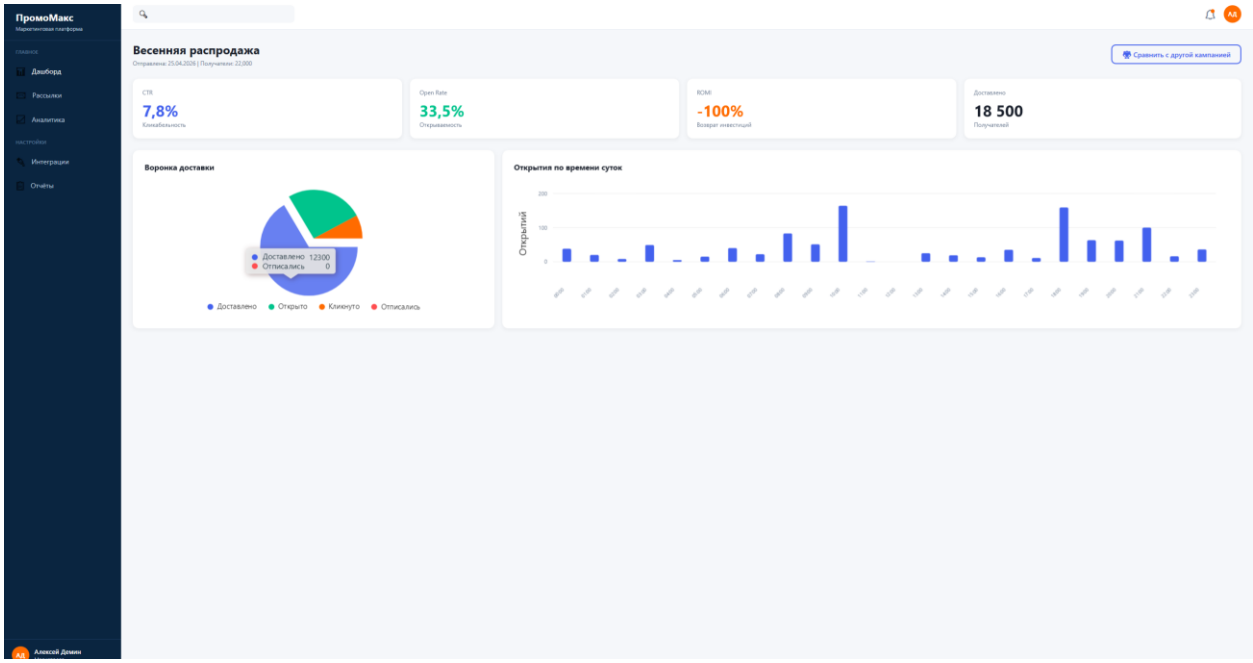

Рисунок 7 – Полный вид аналитики кампании

Скриншот экрана CampaignAnalyticsView: метрики, круговая диаграмма, столбчатый график.

6.4. Настройка интеграций и проверка вебхуков

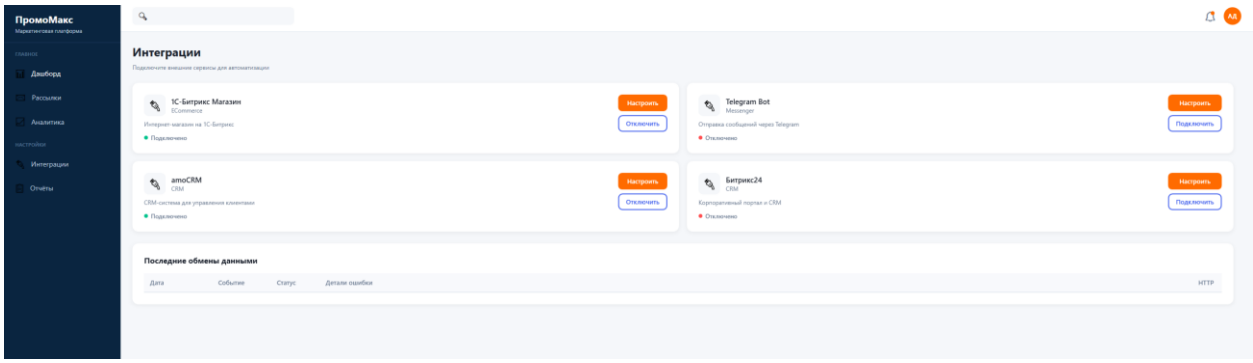

Рисунок 8 – Карточки интеграций

Скриншот: 4 карточки с логотипами, названиями, статусами.

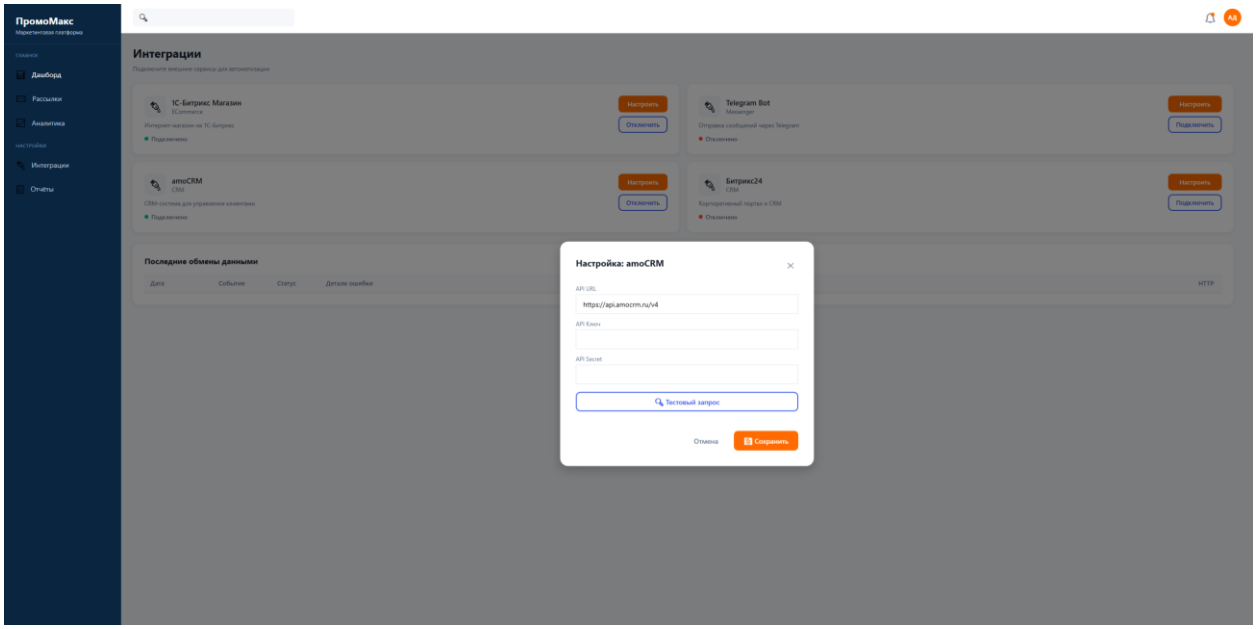

Рисунок 9 – Настройка интеграции и результат тестового запроса

Скриншот модального окна с заполненными полями

6.5. Формирование и экспорт отчётов

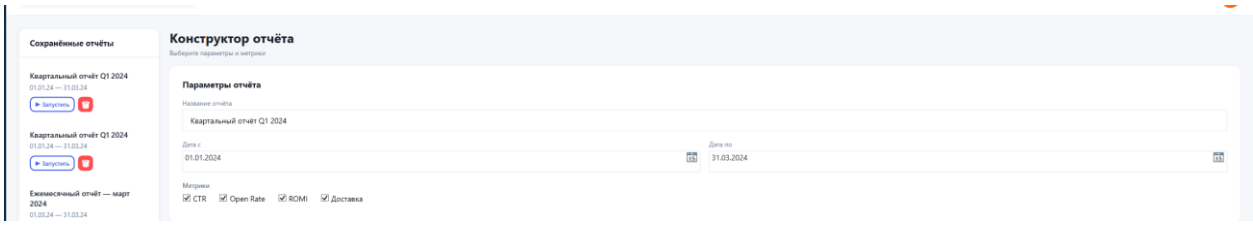

Рисунок 10 – Конструктор отчёта с заполненными параметрами

Выбраны даты, введено название, отмечены метрики.

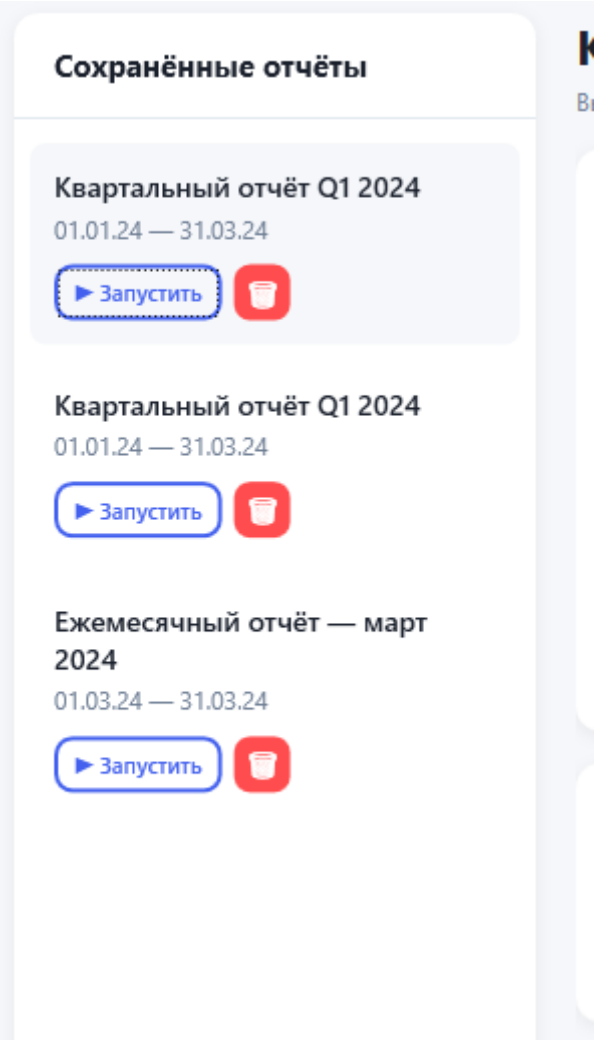

Рисунок 11 – Список сохранённых отчётов

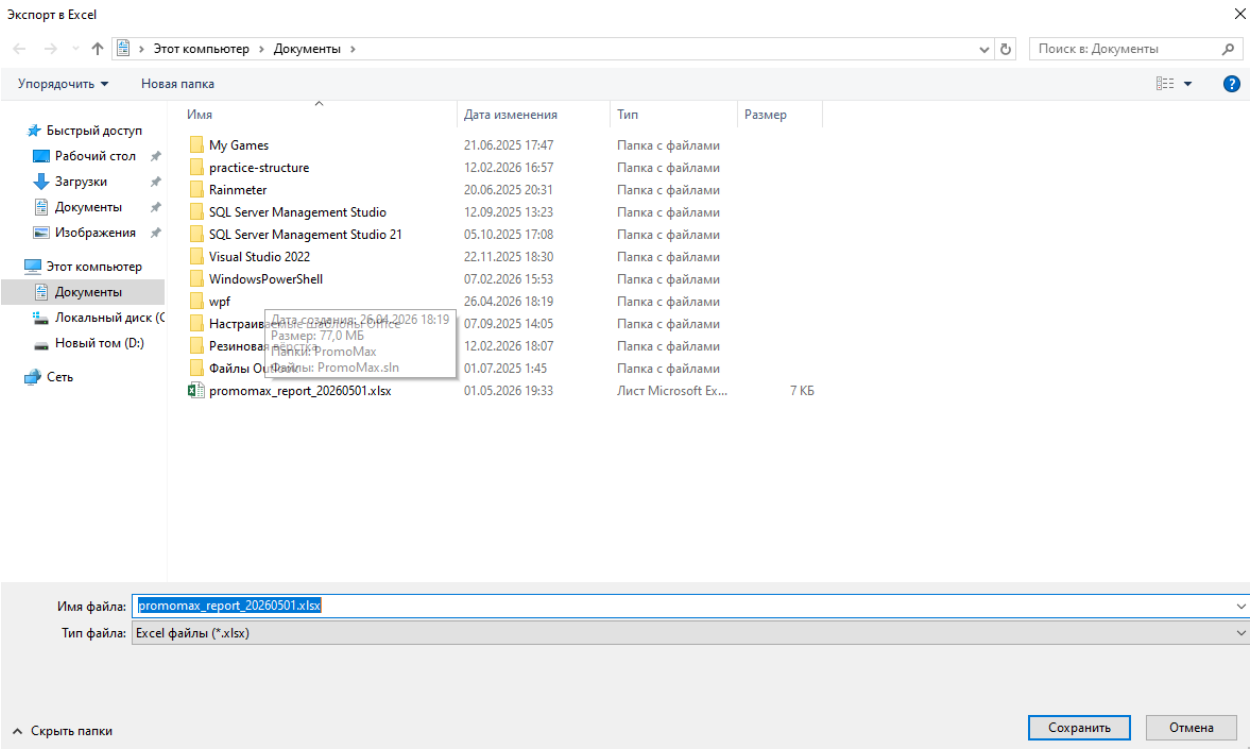

Рисунок 12 – Диалог сохранения Excel-файла

Скриншот стандартного SaveFileDialog с фильтром «Excel файлы (*.xlsx)».

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы по теме «Интеграция модулей маркетинговой платформы “ПромоМакс”» были достигнуты следующие результаты:

1. Разработан полнофункциональный модуль аналитики – реализованы расчёты CTR, ROMI, Open Rate, линейный прогноз тренда (МНК), A/B тестирование. Все данные визуализированы с помощью LiveCharts.
2. Настроен автоматический обмен данными с внешними системами – создан сервис вебхуков с асинхронной отправкой POST-запросов, таймаутами, логированием. API-ключи защищены шифрованием AES-256.
3. Реализованы экспортные возможности – формирование отчётов в PDF (iText7) и Excel (ClosedXML) с профессиональным форматированием.
4. Проведено комплексное тестирование – функциональное, интеграционное, безопасности. Все сценарии пройдены успешно.

Разработанная платформа готова к внедрению и может быть легко расширена для работы с любыми внешними API через настраиваемые вебхуки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Microsoft Corporation. Windows Presentation Foundation (WPF) Documentation [Электронный ресурс]. — Режим доступа: https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/ (дата обращения: 20.04.2025).

2. Microsoft Corporation. .NET 8 Documentation [Электронный ресурс]. — Режим доступа: https://learn.microsoft.com/ru-ru/dotnet/core/whats-new/dotnet-8 (дата обращения: 20.04.2025).

3. Microsoft Corporation. Entity Framework Core — Getting Started [Электронный ресурс]. — Режим доступа: https://learn.microsoft.com/ru-ru/ef/core/get-started/overview/first-app (дата обращения: 21.04.2025).

4. SQLite Documentation [Электронный ресурс]. — Режим доступа: https://www.sqlite.org/docs.html (дата обращения: 21.04.2025).

5. James Wilcock. MaterialDesignInXAML Toolkit [Электронный ресурс]. — Режим доступа: https://github.com/MaterialDesignInXAML/MaterialDesignInXamlToolkit (дата обращения: 22.04.2025).

6. Troelsen A., Japikse P. Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming. — 11th ed. — Apress, 2022. — 1596 p.

7. ГОСТ 19.501-78. Единая система программной документации. Формуляр. Требования к содержанию и оформлению. — М.: Изд-во стандартов, 1978.

8. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2017.